

华中科技大学

课程实验报告

课程名称：大数据分析实验

专业班级 大数据 2301

学 号 U202315458

姓 名 刘严华

指导教师 王蔚

报告日期 2025 年 6 月 1 日

计算机科学与技术学院

目 录

1	实验背景及目的	1
1.1	推荐系统	1
1.2	Minihash 算法	2
1.3	实验目的	2
2	实验内容	3
2.1	任务一：基于用户的协同过滤推荐算法	3
2.2	任务二：基于内容的推荐算法	3
2.3	任务三：使用迷你哈希进行降维	4
3	实验过程	5
3.1	核心算法描述	5
3.2	编程思路	7
3.3	遇到的问题以及解决方式	14
4	实验测试与结果分析	16
4.1	实验环境	16
4.2	实验测试	16
4.3	实验结果分析	17
5	总结展望	21
5.1	总结	21
5.2	展望	21
5.3	课程建议	22
6	附录 A：UUCF 代码	23
7	附录 B：UUCF+Minihash 源代码	32
8	附录 C：CB 源代码	41
9	附录 D：CB+Minihash 源代码	51

1 实验背景及目的

1.1 推荐系统

推荐系统是一种利用用户历史行为、兴趣偏好、社交关系等多维度数据，自动为用户筛选和推送其可能喜欢或需要的信息、商品或服务的信息过滤系统。它能够在海量信息中帮助用户高效发现感兴趣内容，极大提升用户体验，同时为企业带来更高的用户活跃度与商业价值。推荐系统已被广泛应用于电商商品推荐、新闻资讯推送、电影音乐平台等众多场景。

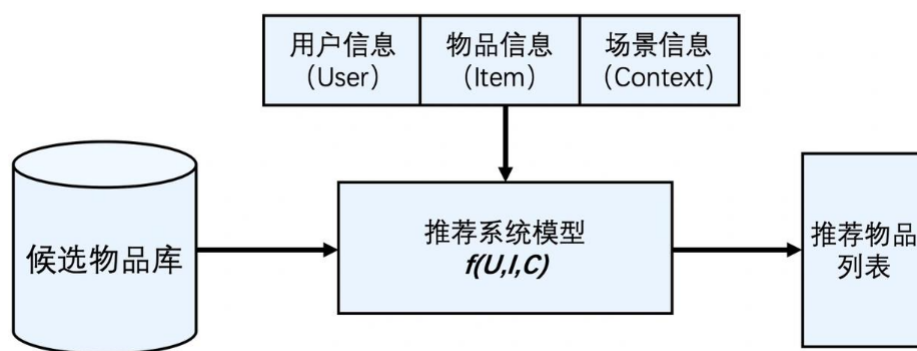


图 1-1 推荐系统简易框架

推荐系统实现的方法多种多样，主要包括以下几类：

1. 协同过滤算法

协同过滤算法。这是最常用的一类推荐算法。该方法通过分析用户之间或用户与物品之间的历史交互行为（如购买、评分、点击等），来挖掘用户偏好和兴趣的相似性，从而进行推荐。协同过滤又可以细分为基于用户的协同过滤和基于物品的协同过滤。其基本思想是，如果两个用户在过去有类似的选择和行为，那么他们对未来项目的偏好也可能相似。例如，如果某用户和你口味相近，他喜欢的电影也有很大概率会被你喜欢。

2. 内容过滤算法

内容过滤算法。该类算法侧重分析项目自身的属性或内容特征，以及用户的兴趣偏好。基于用户过往对某些项目的喜好，提取出项目的特征，并寻找特征相似的其他项目推荐给用户。例如，若某用户喜欢动作题材的电影，内容过滤模型

就会重点推荐具有相似题材或内容特征影片。

3. 混合推荐算法

混合推荐算法将协同过滤和内容过滤等多种方法结合起来，取长补短。通过融合多个模型的推荐结果，可以有效提升推荐的准确性和覆盖度，缓解单一算法存在的局限性。混合推荐在实际大型推荐系统中应用非常广泛。

4. 基于深度学习的推荐算法

随着人工智能技术的发展，深度学习已被广泛应用于推荐系统领域。基于深度学习的推荐算法能够自动从大量数据中学习用户和项目的潜在特征，对复杂的非线性关系建模能力更强。

1.2 Minihash 算法

MiniHash（最小哈希）是一种概率型哈希算法，用于快速估算两个集合之间的 Jaccard 相似度。其核心思想是将集合压缩为短小的“签名”，通过比较签名来近似计算原始集合的相似性，避免直接遍历所有元素，显著降低计算复杂度，尤其适用于大规模数据处理。

1.3 实验目的

- 1、了解推荐系统的多种推荐算法并理解其原理。
- 2、实现 User-User 的协同过滤算法并对用户进行推荐。
- 3、实现基于内容的推荐算法并对用户进行推荐。
- 4、对两个算法进行电影预测评分对比。
- 5、加入 minihash 算法对效用矩阵进行降维处理。

2 实验内容

给定 MovieLens 数据集，包含电影评分，电影标签等文件，其中电影评分文件分为训练集 train_set 和测试集 test_set 两部分。完成以下任务：

2.1 任务一：基于用户的协同过滤推荐算法

对训练集中的评分数据构造用户-电影效用矩阵，使用 pearson 相似度计算方法计算用户之间的相似度，也即相似度矩阵。对单个用户进行推荐时，找到与其最相似的 k 个用户，用这 k 个用户的评分情况对当前用户的所有未评分电影进行评分预测，选取评分最高的 n 个电影进行推荐。预测评分按照以下方式计算：

$$\text{predict_rating} = \frac{\sum_{i=1}^k \text{rating}(i) * \text{sim}(i)}{\sum_{i=1}^k \text{sim}(i)}$$

在测试集中包含 100 条用户-电影评分记录，用于计算推荐算法中预测评分的准确性，对测试集中的每个用户-电影需要计算其预测评分，再和真实评分进行对比，误差计算使用 SSE 误差平方和。

2.2 任务二：基于内容的推荐算法

将数据集 movies.csv 中的电影类别作为特征值，计算这些特征值的 tf-idf 值，得到关于电影与特征值的 n（电影个数）*m（特征值个数）的 tf-idf 特征矩阵。根据得到的 tf-idf 特征矩阵，用余弦相似度的计算方法，得到电影之间的相似度矩阵。

对某个用户-电影进行预测评分时，获取当前用户的已经完成的所有电影的打分，通过电影相似度矩阵获得已打分电影与当前预测电影的相似度，按照下列方式进行打分计算：

$$\text{score} = \frac{\sum_{i=1}^n \text{score}'(i) \cdot \text{sim}(i)}{\sum_{i=1}^n \text{sim}(i)}$$

选取相似度大于零的值进行计算，如果已打分电影与当前预测用户-电影相似度大于零，加入计算集合，否则丢弃。（相似度为负数的，强制设置为 0，表示无相关）假设计算集合中一共有 n 个电影，score 为我们预测的计算结果，score' (i) 为计算集合中第 i 个电影的分数，sim(i) 为第 i 个电影与当前用户-电影的相似度。如果 n 为零，则 score 为该用户所有已打分电影的平均值。

要求能够对指定的 userID 用户进行电影推荐，推荐电影为预测评分排名前 k 的电影。userID 与 k 值可以根据需求做更改。推荐算法准确值的判断：对给出的测试集中对应的用户-电影进行预测评分，输出每一条预测评分，并与真实评分进行对比，误差计算使用 SSE 误差平方和。

2.3 任务三：使用迷你哈希进行降维

主要内容是使用迷你哈希 (MiniHash) 算法对协同过滤算法和基于内容推荐算法的相似度计算进行降维，迷你哈希的模块作为一种近似度的计算方式。

协同过滤算法和基于内容推荐算法都会涉及到相似度的计算，迷你哈希算法在牺牲一定准确度的情况下对相似度进行计算，其能够有效的降低维数，尤其是对大规模稀疏 01 矩阵。可以使用哈希函数或者随机数映射来计算哈希签名。哈希签名可以计算物品之间的相似度。

对于任务一，minihash 采用 jaccard 方法计算相似度，需要对效用矩阵进行 01 处理，也即将 0-2.5 的评分置为 0，2.5-5.0 的评分置为 1。对于任务二，minihash 采用 jaccard 方法计算相似度，特征矩阵应为 01 矩阵，如果该电影存在某特征值，则特征值为 1，不存在则为 0，从而得到 01 特征矩阵。

最终降维后的维数等于我们定义映射函数的数量，我们设置的映射函数越少，整体计算量就越少，但是准确率就越低。大家可以分析不同映射函数数量下，最终结果的准确率有什么差别。

3 实验过程

3.1 核心算法描述

主要采用伪代码来描述三个核心算法

3.1.1 基于用户的协同过滤推荐算法 (User to User Collaborative Filtering)

Require: R : 用户-电影评分矩阵

U : 用户列表

M : 电影列表

k : 邻居数

n : 推荐数

Ensure: 每个用户的 $Top - n$ 推荐电影及预测分数

```
1: // 计算相似矩阵
2: for each pair of users  $u, v$  do
3:    $S[u][v] \leftarrow \text{CalculateSimilarity}(R[u], R[v], \text{sim\_mode})$   ▷ 用 Person 系数计算
   用户之间的相似度
4: end for
5: // 对用户未评分的电影进行评分
6: for each user  $u$  in  $U$  do
7:   for each movie  $i$  in  $M$  do
8:     if  $R[u][i]$  is rated then
9:        $P[u][i] \leftarrow R[u][i]$   ▷  $R[u][i]$  为用户  $u$  对电影  $i$  的评分。如果已评分
       则输出真实值
10:    else
11:       $V \leftarrow \{\text{取评分相似的 } k \text{ 个邻居}\}$ 
12:      for each  $v$  in  $V$  do
13:         $\text{sim}[v] \leftarrow S[u][v]$   ▷ 取邻居的相似度
14:      end for
15:       $G_k \leftarrow \text{Top-}k \text{ users in } V \text{ with highest positive } \text{sim}[v]$ 
16:      if  $\sum_{v \in G_k} |\text{sim}[v]| = 0$  then
```

```
17:          $P[u][i] \leftarrow 0$ 
18:     else
19:          $P[u][i] \leftarrow \frac{\sum_{v \in G_k} \text{sim}[v] \cdot R[v][i]}{\sum_{v \in G_k} |\text{sim}[v]|}$     ▷ 任务一给定的分数计算公式
20:     end if
21: end if
22: end for
23: end for

24: // 推荐  $Top - n$  movies
25: for each user  $u$  in  $U$  do
26:      $U_i \leftarrow$  unrated movies for user  $u$ 
27:     Recommend top- $n$  movies in  $U_i$  sorted by predicted rating  $P[u][i]$ 
28: end for
```

3.1.2 基于内容的推荐算法 (Content-Based)

Require: M : 电影的内容 (如简介、标签等原始文本数据)

U : 用户历史评分数据

k : 需要推荐的电影 K 数目

Ensure: 每个用户未评分电影的预测得分及 top- k 推荐列表

```
// 对  $M$  提取内容特征:
2: for each movie  $m$  in  $M$  do
     $T_m \leftarrow$  Extract content text of movie  $m$  (e.g., synopsis, genre, tags)
4: end for

// 基于所有  $T_m$  用 TF-IDF 方式将每部电影转化为向量
6:  $F \leftarrow$  Calculate TF-IDF vectors for all movies
// 计算所有电影对之间的余弦相似度  $S$ 
8: for each pair of movies  $(i, j)$  do
     $S[i, j] \leftarrow \text{CosineSimilarity}(F[i], F[j])$ 
10: end for

// 根据  $U$  建立用户评分列表
12: for each user  $u$  in  $U$  do
```

```
 $R_u \leftarrow \text{movies rated by user } u$ 
14: for each movie  $m$  not rated by  $u$  do
     $sim \leftarrow$  获取  $m$  与  $R_u$  中各电影的相似度  $\triangleright$  寻找与  $m$  相似的电影
16:     $mask \leftarrow$  indices where  $sim > 0$   $\triangleright$  只取正相似度
    if  $mask$  not empty then
18:         $scores \leftarrow mask$  处的用户评分
         $sims \leftarrow mask$  处的相似度
20:         $predicted\_score[m] \leftarrow \frac{\sum_i scores_i \cdot sims_i}{\sum_i sims_i}$   $\triangleright$  根据给定公式计算预测得分
    else
22:         $predicted\_score[m] \leftarrow 0$ 
    end if
24: end for
    // Top- $k$  的电影得分  $predicted\_score[m]$  , 将其推荐给用户  $u$ 
26: end for
```

3.1.3 Minihash

Require: S : 需要压缩的特征矩阵 k : 哈希函数个数

Ensure: MiniHash 签名矩阵 M , 大小为 $k \times n$

生成 k 个独立的哈希函数 $h_1, h_2, \dots, h_k$

for 每个集合 S_j 在 S 中 **do**

3: **for** 每个哈希函数 h_i , 其中 $i = 1, 2, \dots, k$ **do**

$M[i, j] \leftarrow \min\{h_i(x) : x \in S_j\}$

end for

6: **end for**

输出 MiniHash 后的压缩相似矩阵 M

3.2 编程思路

3.2.1 任务一：基于用户的协同过滤推荐算法

1. 数据分析处理

分析任务需求，需要基于用户内容进行推荐。打开该任务对应的数据训练

集 train_set 和测试集 test_set，观察到 csv 文件主要有 userId、movieId、rating、timestamp 四栏，主要有用的数据是前三栏用户 id, 电影 id, 以及对应的评分。训练集有大概 1M 条数据，其中有许多未评分内容，因此需要构建相对应的用户-电影映射并妥善处理空缺值，这一点在后续遇到的问题中会针对分析。测试集格式与训练集一致，后续需要使用测试集与预测结果比较计算预测性能 SSE 指标。

	A	B	C	D	E
1	userId	movieId	rating	timestamp	
2	1	31	2.5	1260759144	
3	1	1029	3	1260759179	
4	1	1061	3	1260759182	
5	1	1129	2	1260759185	
6	1	1172	4	1260759205	
7	1	1263	2	1260759151	
8	1	1287	2	1260759187	
9	1	1293	2	1260759148	
10	1	1339	3.5	1260759125	
11	1	1343	2	1260759131	
12	1	1371	2.5	1260759135	
13	1	1405	1	1260759203	
14	1	1953	4	1260759191	
15	1	2105	4	1260759139	
16	1	2150	3	1260759194	
17	1	2193	2	1260759198	
18	1	2294	2	1260759108	
19	1	2455	2.5	1260759113	
20	1	2968	1	1260759200	
21	1	3671	3	1260759117	
22	2	10	4	835355493	
23	2	17	5	835355681	
24	2	39	5	835355604	
25	2	47	4	835355552	
26	2	50	4	835355586	
27	2	52	3	835356031	

图 3-1 训练数据

2. 特征矩阵的构建

本次实验构建相似度矩阵主要有以下步骤：首先用 pandas 库按用户 ID、电影 ID 读入矩阵的行列标签—行标签为用户 (UserID)、列标签为电影 (MovieID)，在读入时对其进行重新映射到从 0 开始的下标方便后续处理，然后根据 rating 一栏读入矩阵评分数据元素 (rating_matrix[i,j]) 为用户 i 对电影 j 的评分，未评分位置统一用 0 填充。

```

1 rating_matrix = np.zeros((num_users, num_movies), dtype=np.
    float32)
2 for row in train_df.itertuples():
3     u_idx = user_id2index[row.userId]
4     m_idx = movie_id2index[row.movieId]

```

```
5 rating_matrix[u_idx, m_idx] = row.rating
```

Listing 1 特征矩阵的构建

3. 相似度矩阵构建

为了衡量不同用户对电影的兴趣相似度，采用任务书中所给定的皮尔逊相关系数（Pearson）作为用户间相似度指标。对于任意两个用户，分别取出其评分向量，根据不同的 `sim_mode`（如 ‘overlap’ 共同评分、‘zero’ 补零、‘avg’ 补平均值），决定参与相关系数计算的数据区间。三种模式仅在数据范围上有所区别（通过设立一个 `mask` 决定是否参与计算），计算相似度的方法均为：

$$Pearson_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

```
1 def pearson_sim(self, u1_ratings, u2_ratings):
2     """
3     计算两个用户的皮尔逊相关系数（两向量相似度）。
4     三种方式：
5     - 'zero'      : 未评分用0补齐
6     - 'avg'       : 未评分用该电影的所有评分均值补齐
7     - 'overlap'   : 仅用两人都评分过的电影（最常用，这也是实际
8                     推荐系统最稳妥模式）
9     """
10    if self.sim_mode == 'zero':
11        # 用0补齐，整体对齐
12        x = u1_ratings
13        y = u2_ratings
14        mask = np.ones_like(u1_ratings, dtype=bool)
15    elif self.sim_mode == 'avg':
16        avg = self.movie_means
17        x = u1_ratings.copy()
18        y = u2_ratings.copy()
19        x[x == 0] = avg[x == 0] # 用电影的平均分补空
20        y[y == 0] = avg[y == 0]
21        mask = np.ones_like(u1_ratings, dtype=bool)
22    else: # 'overlap'
23        # 只考虑两人都打过的电影
24        mask = (u1_ratings > 0) & (u2_ratings > 0)
```

```
24         if np.sum(mask) < 2:      # 如果没有或仅有1部重合, 返回0
                                   , 防止过拟合
25             return 0.0
26         x = u1_ratings[mask]
27         y = u2_ratings[mask]
28         if np.sum(mask) < 2:
29             return 0.0
30         mean_x = np.mean(x)
31         mean_y = np.mean(y)
32         numerator = np.sum((x - mean_x)*(y - mean_y))
33         denominator = np.sqrt(np.sum((x - mean_x)**2)) * np.sqrt(
                                   np.sum((y - mean_y)**2))
34         if denominator == 0:
35             return 0.0 #防止零除
36         return numerator / denominator
```

Listing 2 pearson 系数计算

4. 推荐评分

要给用户未评分电影评分, 因此需要遍历每位用户, 对其未评分的每一部电影进行评分预测:

针对某一用户的一部未评分电影, 先找出所有评分过该电影的“邻居用户”;

从这些邻居中, 基于相似度挑选 Top-K 个最相关用户;

仅利用正相关(兴趣倾向相同)的邻居, 根据他们对该电影的评分与相似度加权平均, 预测目标用户可能的评分。

最后在预测的评分列表中选出预测得分最高的 n 部电影(如 n=5), 作为最终推荐列表。

```
1 for n_idx, sim in zip(neighbors, neighbors_sims):
2     if sim <= 0: continue # 只用正向相似邻居
3     numerator += sim * self.rating_matrix[n_idx, m_idx]
4     denominator += abs(sim)
```

Listing 3 推荐评分计算

3.2.2 任务二：基于内容的推荐算法

1. 数据分析处理分析任务需求，数据训练集 `train_set` 和测试集 `test_set` 的构成不再赘述，任务二相较于任务一多了一个 `movies.csv`，其中有 `movieId`、`title`、`genres`，相较于任务一相似矩阵构建不是基于相近用户而是基于内容本身，因此在下一步特征矩阵构建时与任务一有较大出入。

1	movieId	title	genres
2	1	Toy Story (1995)	Adventure Animation Children Comedy Fantasy
3	2	Jumanji (1995)	Adventure Children Fantasy
4	3	Grumpier Old Men (1995)	Comedy Romance
5	4	Waiting to Exhale (1995)	Comedy Drama Romance
6	5	Father of the Bride Part II (1995)	Comedy
7	6	Heat (1995)	Action Crime Thriller
8	7	Sabrina (1995)	Comedy Romance
9	8	Tom and Huck (1995)	Adventure Children
10	9	Sudden Death (1995)	Action
11	10	GoldenEye (1995)	Action Adventure Thriller
12	11	American President, The (1995)	Comedy Drama Romance
13	12	Dracula: Dead and Loving It (1995)	Comedy Horror
14	13	Balto (1995)	Adventure Animation Children
15	14	Nixon (1995)	Drama
16	15	Cutthroat Island (1995)	Action Adventure Romance
17	16	Casino (1995)	Crime Drama
18	17	Sense and Sensibility (1995)	Drama Romance
19	18	Four Rooms (1995)	Comedy
20	19	Ace Ventura: When Nature Calls (1995)	Comedy
21	20	Money Train (1995)	Action Comedy Crime Drama Thriller
22	21	Get Shorty (1995)	Comedy Crime Thriller
23	22	Copycat (1995)	Crime Drama Horror Mystery Thriller
24	23	Assassins (1995)	Action Crime Thriller
25	24	Powder (1995)	Drama Sci-Fi
26	25	Leaving Las Vegas (1995)	Drama Romance
27	26	Othello (1995)	Drama
28	27	Now and Then (1995)	Children Drama
29	28	Persuasion (1995)	Drama Romance
30	29	City of Lost Children, La Cité des Enfants Perdus (1995)	Adventure Drama Fantasy Mystery Sci-Fi
31	30	Shanghai Triad (Yao a Hei) (1995)	Crime Drama
32	31	Dangerous Minds (1995)	Drama
33	32	Twelve Monkeys (a.k.a. The Year of the Living Dead) (1995)	Mystery Sci-Fi Thriller

图 3-2 电影数据

2. 特征矩阵的构建 (tf-idf) 采用任务书中 TF-IDF (词频-逆文档频率) 方法基于电影类别 (genres) 构建电影的特征矩阵。TF-IDF 处理过程如下：

首先，统计所有电影出现过的类型总集合 U ；

然后对每部电影，统计其涉及的类型（如 Action|Adventure），给每个类型赋予对应的 TF-IDF 值：

$$TF-IDF_{i,j} = 1 + \log \left(\frac{1 + N}{1 + n_j} \right)$$

其中 N 为电影总数, n_j 第 j 种类型出现的电影数。这样稀有类型权重更高。

得到一个 $M \times C$ 的特征矩阵 (M 为电影数, C 为类型种数) 按行归一化后用于后续相似度计算。

```
1 tfidfMat[i][mapper[word]] = 1 + math.log((1 + length) / (1 + self
    .categoryAppearSum[word]))
2 # 行归一化
3 tfidfMat[i] = np.divide(tfidfMat[i], np.sqrt(np.sum(np.square(
    tfidfMat[i]))))
```

Listing 4 TF-IDF 计算

3. 相似度矩阵构建

基于内容的相似度采用余弦相似度:

首先获得所有电影的 TF-IDF 向量 (特征矩阵)。

对任意两部电影, 计算其 TF-IDF 向量的余弦相似度, 结果为

$M \times M$ 对称矩阵, 元素为 $[0,1]$ 区间。

```
1 similarityMatrix = processor.getCosSimilarBetweenTwoMat(tfidfMat,
    tfidfMat)
```

Listing 5 相似度计算

4. 推荐评分

找出用户历史评分过的所有电影。分别计算未评分电影与这些已评分电影的内容相似度。加权平均用户的历史评分, 权重为内容相似度, 只考虑正相似度部分:

```
1 positive_similarities = similarities[positive_mask]
2 positive_scores = user_scores[positive_mask]
3 return np.sum(positive_scores * positive_similarities) / np.sum(
    positive_similarities)
```

Listing 6 推荐评分计算

3.2.3 任务三：使用迷你哈希进行降维

1) UUCF+Minihash

与普通 uucf 唯一的区别是在相似度计算采用了 minihash+jarcard 方法计算相似度, 并按照任务书要求对效用矩阵进行 01 处理, 也即将 0-2.5 的评分置为 0, 2.5-5.0 的评分置为 1。在评分算法上与没有 Minihash 版本并无二异, 只对 Minihash 部分进行分析:

1. Minihash 函数生成

随机产生 N 组参数 (a_i, b_i) 用于构建 hash 函数:

$$h_i(x) = (a_i x + b_i) \bmod p$$

2, hash 过程: 基于每一部电影的 TF-IDF 特征构建 01 向量, 提取所有为 1 的位置 (代表该对象包含的特征), 对于每个哈希函数, 在集合所有元素编号上分别计算哈希值, 记录最小哈希结果作为当前签名的一个分量, 然后汇总 n 个分签名构成该电影的对象的最小哈希签名。

```
1 ones_positions = np.where(binary_vector)[0]
2 for i in range(self.n_hash_functions):
3     hash_values = [(self.a[i] * x + self.b[i]) % self.prime for x
4                     in ones_positions]
5     min_hash = min(hash_values) if hash_values else float('inf')
6     signature.append(min_hash)
```

Listing 7 hash 过程

2) CB+Minihash

CB 的 minihash 过程与 UUCF 类似, 按照任务书要求特征矩阵应为 01 矩阵, 如果该电影存在某特征值, 则特征值为 1, 不存在则为 0, 从而得到 01 特征矩阵。在此也只对 Minihash 计算相似度部分进行分析: 因为调用了 datasketch 库的 Minhash 算法, 可以直接指定 hash 函数个数并得出最后的签名。

```
1 for row in binaryMat:
2     m = MinHash(num_perm=self.num_perm)
3     for idx, v in enumerate(row):
4         if v == 1:
5             m.update(str(self.categoryList[idx]).encode('utf8'))
```

```
6 minhashes.append(m)
```

Listing 8 调库的 minihash

3.3 遇到的问题以及解决方式

3.3.1 基于用户推荐中缺失评分的处理

在 UUCF 数据分析处理过程中，任务书和 PPT 并未明确给出对于空缺值是如何处理的，经过搜索查询后找到了三种不同的填充模式，因此设立了一个 `sim_mode` 参数供选择。三者的主要处理方式以及资料中所给出的优缺点如下表：

模式	填充策略说明	优点	缺点
zero	未评分部分补0，对齐全维度比较	1. 代码实现简单	1. 0并不代表真实评分，偏向注入大量“负反馈”
		2. 覆盖全集数据，内存对齐	2. 稀疏度高时噪声大，低估相似度
		3. 对新用户推荐电影较难	
avg	未评分部分补该电影所有用户的平均分	1. 缓解0的极端影响，较“零填充”更贴近实际	1. 平均值可能隐藏用户真实偏好差异
			2. 与用户偏好不一定相符
overlap	只对两用户都评分过的电影计算相似度	1. 最反映真实相似度，忽略不可比项	1. 新用户/电影交集样本少时相似度缺失
		2. 避免“无交集”误差	2. 有时无法为冷启动做出响应

图 3-3 三种填充对比

3.3.2 minihash 的哈希函数实现

同样 minhash 只是一个大致算法框架，hash 函数是如何生成以及 hash 函数数量选择决定权在我们手上，在任务一和任务二我分别采用了自己编写算法生成 hash 函数并进行 Minhash 过程和调用 `datasketch` 库中已有的 Minhash 算法进行计算两种不同的实现方式。

3.3.3 推荐结果异常处理

在任务一进行推荐时发现最终的推荐结果全部一样，发现是由于处理数据进行了映射但是评分推荐完成后没有根据索引还原 ID。

3.3.4 重复计算导致时间消耗过长

运行代码时经常需要跑好几分钟，为了避免重复计算相似矩阵，引入了 `joblib` 以二进制保存相似矩阵文件，后续进行计算时可以直接复用已经计算好的相似矩阵，从而大大减少计算的等待时间。

```
1 similarity_matrix_path = 'similarity_matrix.joblib'
2 if os.path.exists(similarity_matrix_path):
3     print("加载已保存的相似度矩阵...")
4     similarityMatrix = joblib.load(similarity_matrix_path) # 加
        载已有特征
5 else:
6     print("计算电影相似度矩阵（这可能需要一些时间）...")
7     processor = moviePreprocess(df1)
8     similarityMatrix = processor.getSimilarityMat() # 重新计算内
        容相似度
9     print("保存相似度矩阵到文件以便后续使用...")
10    joblib.dump(similarityMatrix, similarity_matrix_path) # 缓
        存，便于下次直接用
```

Listing 9 使用 joblib 保存相似矩阵

4 实验测试与结果分析

4.1 实验环境

1. 硬件环境:

主板: ASUS FA507XV

CPU: AMD Ryzen 9 7940H

RAM: Crucial Technology DDR5 5600Mhz 16GB*2

2. 软件环境:

OS: Windows11 23H2

IDE: Visual Studio Code

Python 版本: 3.12.3

4.2 实验测试

本实验总计有四个 Python 文件, USER 目录下有 U1.py 和 U1pro.py 文件分别是 UCCF 和 UCCF+Minihash 的实验代码; MOVIE 目录下有 M1.py 和 M1promax.py 文件分别是 CB 和 CB+Minihash 的实验代码。

运行代码需要打开指定文件夹(请勿修改 dataset 位置), 然后打开需要运行的 python 代码文件确保具备相应的 python 库后点击运行即可, 运行过程控制台有文字提示任务进度。任务一可以设定推荐参数邻居数 k (默认为 10) 和推荐电影数目 n (默认为 5)。任务一和二可以通过修改 hash 部分参数指定 hash 函数个数。

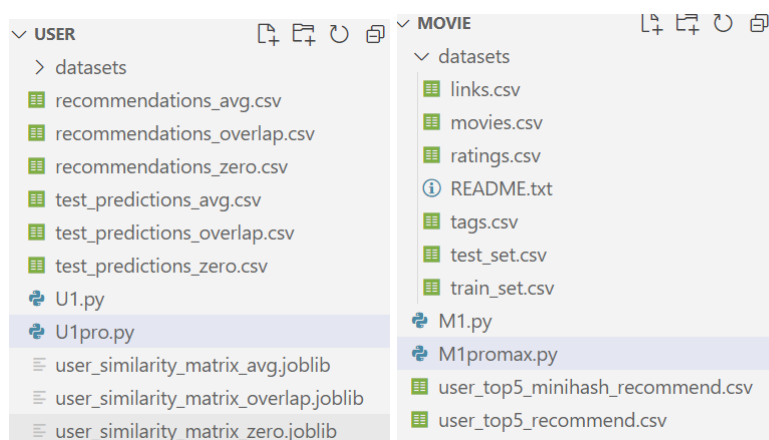


图 4-1 文件目录

预测评分与真实值对比以及推荐结果会自动保存到 csv 文件内可以打开查看。

```

recommendations_avg.csv > | data
1 userId,ReelMovieId_1,Score_1,ReelMovieId_2,Score_2,ReelMovieId_3,Score_3,ReelMovieId_4,Score_4,ReelMovieId_5,Score_5
2 1,163949,5.0,1543,5.0,3838,5.0,3821,5.0,118898,5.0
3 2,163949,5.0,6187,5.0,3112,5.0,27792,5.0,5968,5.0
4 3,163949,5.0,4281,5.0,5871,5.0,5862,5.0,4945,5.0
5 4,163949,5.0,1409,5.0,4835,5.0,1575,5.0,1565,5.0
6 5,163949,5.0,48682,5.0,67584,5.0,26581,5.0,5427,5.0
7 6,163949,5.0,5473,5.0,118873,5.0,5381,5.0,5264,5.0
8 7,163949,5.0,58861,5.0,3975,5.0,5388,5.0,3611,5.0
9 8,163949,5.0,4565,5.0,4789,5.0,4755,5.0,4731,5.0
10 9,134449,5.0,4584,5.0,9324,5.0,9320,5.0,3879,5.0
11 10,163949,5.0,59392,5.0,3216,5.0,3282,5.0,3173,5.0
12 11,163949,5.0,114464,5.0,3968,5.0,3765,5.0,3473,5.0
13 12,163949,5.0,2649,5.0,7382,5.0,7355,5.0,7564,5.0
14 13,163949,5.0,47728,5.0,54251,5.0,53887,5.0,52617,5.0
15 14,163949,5.0,1455,5.0,5427,5.0,3473,5.0,3765,5.0
16 15,163949,5.0,114464,5.0,53887,5.0,52617,5.0,754,5.0
17 16,163949,5.0,127852,5.0,26749,5.0,129313,5.0,26599,5.0
18 17,163949,5.0,4382,5.0,96832,5.0,4281,5.0,37857,5.0
19 18,163949,5.0,4786,5.0,5745,5.0,3473,5.0,5427,5.0
20 19,163949,5.0,47728,5.0,3656,5.0,26787,5.0,3612,5.0
21 20,2982,5.0,5264,5.0,559,5.0,25881,5.0,15852,5.0
22 21,163949,5.0,4442,5.0,4724,5.0,129313,5.0,183731,5.0
23 22,163949,5.0,4342,5.0,4382,5.0,4252,5.0,96887,5.0
24 23,163949,5.0,3588,5.0,3746,5.0,48226,5.0,3656,5.0
25 24,163949,5.0,4711,5.0,5181,5.0,5071,5.0,5862,5.0
26 25,163949,5.0,57836,5.0,4888,5.0,121126,5.0,4076,5.0
27 26,69761,5.0,61258,5.0,4591,5.0,68384,5.0,68674,5.0
28 27,163949,5.0,27592,5.0,26797,5.0,26791,5.0,26749,5.0
29 28,163949,5.0,6725,5.0,5865,5.0,58613,5.0,26587,5.0
30 29,163949,5.0,3112,5.0,31413,5.0,3838,5.0,3821,5.0
31 30,163949,5.0,6332,5.0,7268,5.0,114464,5.0,7288,5.0
32 31,163949,5.0,3165,5.0,3588,5.0,3575,5.0,58783,5.0

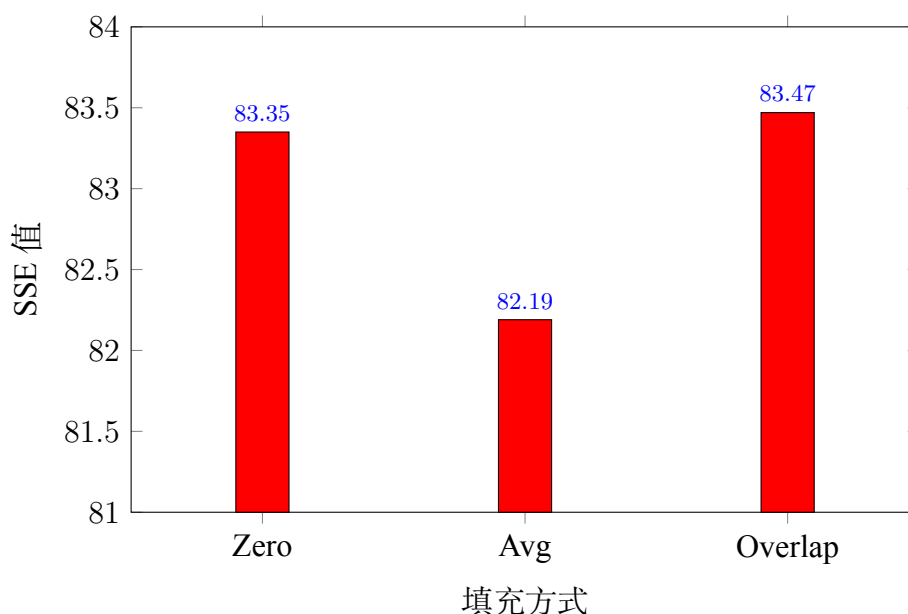
test_predictions_avg.csv > | data
1 userId,ReelMovieId,True_Rating,predicted_rating
2 547,2,3,3,3.9999852
3 547,2,5,3.7999892
4 564,1,4,3.8499629
5 564,2,4,3.149986
6 624,1,5,4.2888855
7 624,2,3,3.2498895
8 15,1,2,0,3.4999971
9 15,2,2,0,2.8998836
10 73,1,5,3.9998596
11 73,2,2,3,4.497297
12 452,1,3,5,4.1499815
13 452,2,1,3,3.99967
14 468,1,4,3.9999948
15 468,2,2,3,0.8498104
16 388,2,4,4,2.49959
17 388,4,4,3.9888328
18 311,1,3,4,3.888255
19 311,4,4,3.7998374
20 38,1,4,4,2.999835
21 38,2,2,3,3.1498385
22 294,1,4,4,2.849854
23 294,5,3,3,3.888846
24 589,1,3,4,2.99955
25 589,2,2,3,4.499646
26 589,4,4,3.999586
27 589,2,3,3,0.8497487
28 213,1,3,3,3.658826
29 213,2,3,3,3.4488768
30 212,2,3,3,3.1599717
31 212,2,3,3,3.349714
32 472,1,5,4,3.8888333
    
```

图 4-2 输出的 csv 文件

4.3 实验结果分析

4.3.1 UUCF 空值不同处理方式对预测性能的影响分析

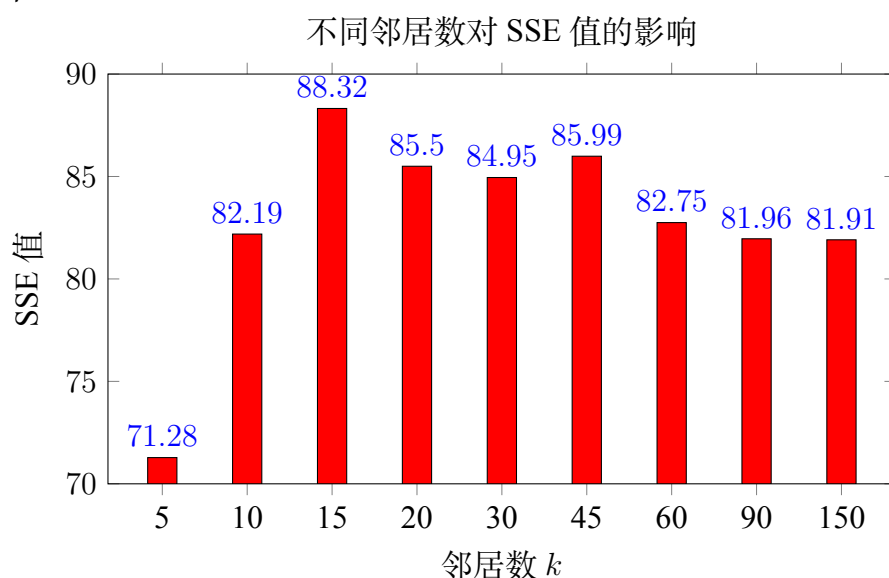
不同填充方式对预测性能的影响



根据图片可以得出在本实验的数据集条件下，采用 Avg（均值）填充方式能够有效降低 SSE，提高模型预测性能，优于 Zero 和 Overlap 两种方式。因此在后续分析中采用 avg 填充。

4.3.2 UUCF 中邻居个数的对预测结果的影响分析 (avg 填充)

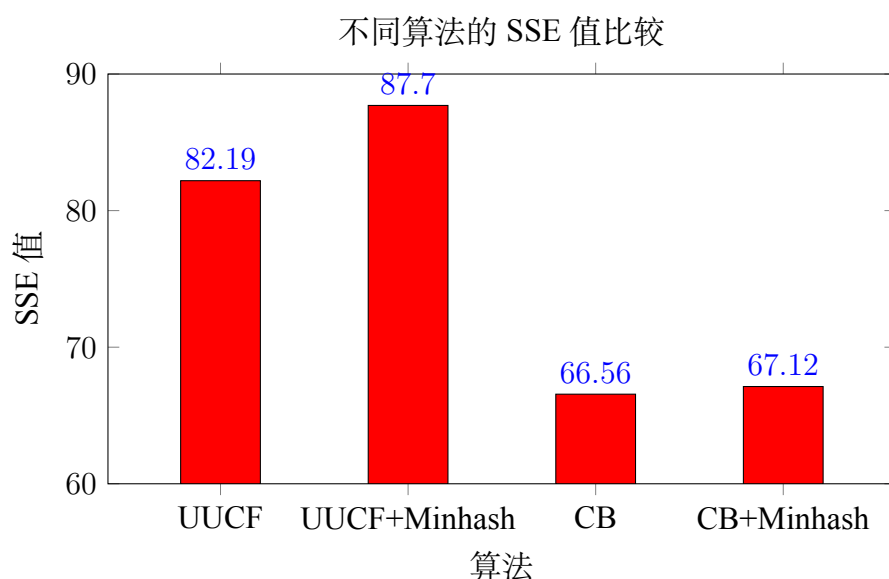
在 UUCF 无 minihash 和均值填充 (avg) 的前提下改变 k 值绘制的 sse 条形图如下:



从图中可以看到,随着邻居数 k 的增加, SSE 值的变化呈现出先增大后减小的趋势。当邻居数较小时 (如 $k=5$), SSE 值达到最小,说明此时预测效果较好。随着 k 的不断增大, SSE 值逐步上升,并在 $k=15$ 处达到最大值,表明过多的邻居会引入噪声,导致预测误差增大。当进一步增大时, SSE 值又开始下降,并逐渐趋于稳定,但整体再未低于 $k=5$ 时的最小值。这一现象说明,在基于用户协同过滤的推荐算法中,邻居数的选择对推荐效果有重要影响。较小的邻居数有利于提升预测准确性,但过小可能导致信息利用不足;过大的邻居数则易引入无关信息,使得误差增加。选择合适的邻居数可以在准确性与实用性之间取得较好平衡。

4.3.3 Minihash 算法对对预测结果的影响

1. 没有 Minihash 和有 Minihash 算法在预测性能对比



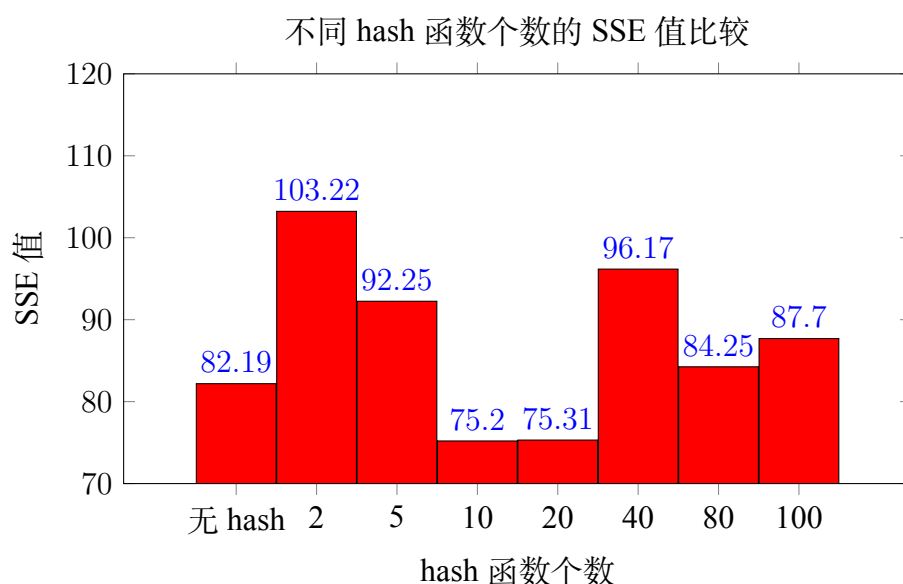
通过上图不同算法的 SSE 值对比，我们可以得到如下结论：

(1) 基于内容的推荐算法 CB 的预测性能优于基于用户的协同过滤算法 UUCF。从图中可以看到，CB 算法的 SSE 值为 66.56，显著小于 UUCF 算法的 82.19。说明在本实验数据条件下，CB 算法能够更准确地进行预测，这一结论与课堂上所讲内容一致。

(2) 引入 Minhash 会带来预测性能的损失。在两种基础算法中引入 Minhash 后，SSE 值均有所上升，性能均有所下降。具体来说，UUCF+Minhash 的 SSE 为 87.7，高于原 UUCF 的 82.19；CB+Minhash 的 SSE 为 67.12，也稍高于原 CB 的 66.56。说明 Minhash 在提升运算效率和降维的同时，会带来一定的性能损失。

(3) 自定义与调库的 Minhash 方法对性能损失的影响不同。UUCF 采用自己生成的 hash 函数实现 Minhash，CB 则调用了现有库的 Minhash 方法。由图比较可发现，UUCF 加入 Minhash 后 SSE 提升较多（从 82.19 升至 87.7），而 CB 加入 Minhash 后 SSE 仅略有上升（从 66.56 升至 67.12）。这说明调库的 Minhash 方法在预测性能方面表现要优于自己编写的 minhash 实现。

2.Hash 函数个数不同时的预测性能对比（以任务一为例）



观察发现：

(1) 当 hash 函数个数较少时（如 2 个、5 个），引入 hash 会使 SSE 值明显上升，预测性能变差。例如，2 个 hash 函数时 SSE 高达 103.22，远高于无 hash 时的 82.19。

(2) 但当 hash 函数个数增加到一定数目（10 和 20 时），SSE 值显著下降，分别为 75.2 和 75.31，甚至优于未使用 hash 时的表现，可能是因为此时 hash 特征更全面，能更好地区分不同用户或内容。

(3) 但继续增大 hash 函数个数（如 40、80、100）后，性能提升并未持续，反而有回落现象。从图中可以看出，SSE 值在 hash 函数为 40 个时反弹至 96.17，80 个时为 84.25，100 个时为 87.7，均高于 10 和 20 个时的 SSE 值，推测是过多的 hash 函数可能引入冗余、噪声等负面影响导致预测性能降低。

综合上述分析可知过多 hash 函数和过少 hash 函数个数都会带来预测性能的损失，因此需要选定合适的 hash 函数个数才能兼顾预测速度与预测性能。

5 总结展望

5.1 总结

(1) 本次实验设定的三个主要任务均已顺利完成，预期的实验目标基本实现，实验结果也符合预期以及课堂讲授知识，通过了实验验收，实验设计预期良好。

(2) 学习并掌握了推荐系统中的两种基本方法：基于用户的协同过滤 (User-based Collaborative Filtering, UUCF) 和基于内容的推荐算法 (Content-based, CB)，同时对降维算法 Minhash 的基本原理及其在实际中的 Python 代码实现有了基本了解，除此之外，实验让我对于大数据的分析、处理、总结流程有了总体观念，为以后大数据深入的学习奠定了一定基础。

(3) 在实验过程中，遇到了一些实际问题与挑战，如数据预处理、参数选择、算法实现细节等。通过查阅资料、尝试不同方案并动手调试，成功解决了这些问题，个人的知识储备和编程能力得到了显著提升。

(4) 实验结束后，对模型的预测结果和相关性能指标（如 SSE 值）进行了较为全面的分析，总结出了一些积极的结论。例如，不同推荐算法的性能差异、Minhash 降维带来的影响，以及 hash 参数选择对性能的具体作用等。这些实验后的复盘总结让我对推荐系统的原理有了更深的理解。

5.2 展望

(1) 本次实验仅复刻实现了基于用户协同过滤和基于内容推荐这两种经典的推荐系统算法。通过本次实践，对推荐系统的基本原理和实现流程有了深入了解，但也意识到当前主流推荐算法日新月异。未来希望能够继续深入学习包括深度学习推荐算法在内的其他先进技术，例如基于神经网络的协同过滤、图神经网络推荐等。

(2) 在实验过程中，发现当前代码在运算效率和准确度方面仍有较大提升空间。在数据量较大时，运行结果需要等待数分钟才能输出。虽然采用了 joblib 对相似矩阵进行保存，大大缩短了相似矩阵的重复计算时间，但最终推荐部分仍然是耗时的主要环节。未来可以从算法优化、数据结构改进、并行计算等方面进一步提升系统的时间性能。

(3) 本次实验的数据来源和特征选择较为有限，仅作为实验是可以的，距离

真正日常生活中所遇到的推荐系统还有较大距离，未来可以考虑更加贴近实际应用的推荐系统设计，开发可用于实际的推荐系统。

(4) 本学期大数据分析课程到此圆满结束，通过实验我对大数据基础理论及其实际应用有了更加系统与全面的认识。然而，作为大数据专业的学生，深刻体会到大数据知识体系极为庞杂且不断发展。大数据分析课程的学习只是一个起点，未来仍需不断自我充电和实践积累，希望通过今后系列大数据课程的学习和项目实践，进一步提升数据分析能力与工程实现水平，为今后在大数据领域的深造与发展打下坚实基础。

5.3 课程建议

5.3.1 补充实验实现的部分细节

在实验时经常有一种很多选择的茫然的感觉，在做小实验 PageRank 时因为 dead-end 处理到底是直接保持某一系列全为 0 仅作防止零除处理还是用 $1/n$ 填充并无明确的要求，只能从验收时的结果存在出入反推应该是前者；在做 wordcount 时对于分布数据不存在内存里这一要求一开始也是理解不到位，在第一次验收助教的指引下才通过分步保存当前片文件达到了这一要求，可以在以后的任务书中明确一下验收的具体技术要求。

5.3.2 可以设置反馈问卷

课程可以每隔几周开展一次阶段性反馈，例如通过匿名问卷或在线调查，收集同学们对课堂进度、教学内容、课程实验等方面的反馈意见。针对集中提出的问题，这样可以提升教学的针对性和实效性，同时也帮助老师自身不断优化教学方法。

感谢您看到这里，老师辛苦了！祝大数据分析课程越来越好！

6 附录 A: UUCF 代码

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import math
6 import warnings
7 from joblib import dump, load
8
9 warnings.filterwarnings("ignore")
10
11 # ===== 1. 参数设置 =====
12 k_neighbors = 10      # 选择最相似的Top-K邻居数量
13 n_movies = 5          # 每个用户最终推荐的电影数
14 sim_mode = 'overlap'  # 相似度计算方式: 'zero'=全补零, 'avg'=补电
                        # 影均值, 'overlap'=共同评分 (最常用、最准确)
15
16 # ===== 2. 数据加载与预处理 =====
17 # 读入训练集和测试集
18 train_df = pd.read_csv("datasets/train_set.csv") # 样例格式:
                        #   userId, movieId, rating
19 test_df = pd.read_csv("datasets/test_set.csv")  # 用于最终评测
20
21 # 收集所有用户与电影的id, 保证id和矩阵下标0基序一致
22 all_user_ids = sorted(train_df['userId'].unique())
23 all_movie_ids = sorted(train_df['movieId'].unique())
24
25 # 将用户/电影id映射为下标 (便于做矩阵运算)
26 user_id2index = {u: idx for idx, u in enumerate(all_user_ids)}
27 movie_id2index = {m: idx for idx, m in enumerate(all_movie_ids)}
28 num_users = len(all_user_ids)
29 num_movies = len(all_movie_ids)
30 print(f"用户数量: {num_users}")
31 print(f"电影数量: {num_movies}")
```

```
32
33 # 初始化用户-电影评分矩阵，用户为行，电影为列，未评分位置为0
34 rating_matrix = np.zeros((num_users, num_movies), dtype=np.
    float32)
35 for row in train_df.itertuples():
36     u_idx = user_id2index[row.userId]
37     m_idx = movie_id2index[row.movieId]
38     rating_matrix[u_idx, m_idx] = row.rating
39
40 # ===== 3. 推荐系统主类 =====
41 class MovieRecommender:
42     """
43     基于用户-用户协同过滤的电影推荐系统
44     """
45     def __init__(self, rating_matrix, user_id2index,
        movie_id2index, sim_mode):
46         """
47         rating_matrix: 用户-电影评分矩阵
48         sim_mode:      相似度计算模式('zero', 'avg', 'overlap')
49         """
50         self.rating_matrix = rating_matrix
51         self.user_id2index = user_id2index
52         self.movie_id2index = movie_id2index
53         self.sim_mode = sim_mode # 相似度计算模式
54         self.user_sim_matrix = None # 用户相似度矩阵（懒加载/自动
            缓存）
55         self.matrix_file = f'user_similarity_matrix_{self.
            sim_mode}.joblib' # 用户相似度矩阵缓存文件
56         # 下标与ID的反查
57         self.index_to_movie = {idx: movie_id for movie_id, idx in
            movie_id2index.items()}
58         self.index_to_user = {idx: user_id for user_id, idx in
            user_id2index.items()}
59         # 统计每部电影的均值（补平均值模式需要）
60         self.movie_means = np.zeros(rating_matrix.shape[1], dtype
```

```
        =np.float32)
61     for m_idx in range(rating_matrix.shape[1]):
62         scores = rating_matrix[:, m_idx]
63         scored = scores[scores > 0]
64         self.movie_means[m_idx] = np.mean(scored) if len(
            scored) > 0 else 0
65
66     def pearson_sim(self, u1_ratings, u2_ratings):
67         """
68         计算两个用户的皮尔逊相关系数（两向量相似度）。
69         三种方式：
70         - 'zero'      : 未评分用0补齐
71         - 'avg'       : 未评分用该电影的所有评分均值补齐
72         - 'overlap'   : 仅用两人都评分过的电影（最常用，这也是实际
            推荐系统最稳妥模式）
73         """
74         if self.sim_mode == 'zero':
75             # 用0补齐，整体对齐
76             x = u1_ratings
77             y = u2_ratings
78             mask = np.ones_like(u1_ratings, dtype=bool)
79         elif self.sim_mode == 'avg':
80             avg = self.movie_means
81             x = u1_ratings.copy()
82             y = u2_ratings.copy()
83             x[x == 0] = avg[x == 0] # 用电影的平均分补空
84             y[y == 0] = avg[y == 0]
85             mask = np.ones_like(u1_ratings, dtype=bool)
86         else: # 'overlap'
87             # 只考虑两人都打过的电影
88             mask = (u1_ratings > 0) & (u2_ratings > 0)
89             if np.sum(mask) < 2: # 如果没有或仅有1部重合，返回0
                , 防止过拟合
90                 return 0.0
91             x = u1_ratings[mask]
```

```
92         y = u2_ratings[mask]
93     if np.sum(mask) < 2:
94         return 0.0
95     mean_x = np.mean(x)
96     mean_y = np.mean(y)
97     numerator = np.sum((x - mean_x)*(y - mean_y))
98     denominator = np.sqrt(np.sum((x - mean_x)**2)) * np.sqrt(
99         np.sum((y - mean_y)**2))
100     if denominator == 0:
101         return 0.0 #防止零除
102     return numerator / denominator
103
104 def compute_similarity_matrix(self):
105     """
106     计算所有用户两两的相似度矩阵，若有缓存直接读取，否则计算
107     后缓存
108     """
109     if os.path.exists(self.matrix_file):
110         print(f"从缓存加载用户相似度矩阵 ({self.sim_mode}模
111             式) ...")
112         self.user_sim_matrix = load(self.matrix_file)
113         return
114     print(f"计算用户相似度矩阵 ({self.sim_mode}模式) ...")
115     num_users = self.rating_matrix.shape[0]
116     self.user_sim_matrix = np.zeros((num_users, num_users),
117         dtype=np.float32)
118     for i in range(num_users):
119         if i % 100 == 0:
120             print(f"计算进度: {i}/{num_users}")
121         for j in range(i + 1, num_users):
122             sim = self.pearson_sim(self.rating_matrix[i],
123                 self.rating_matrix[j])
124             self.user_sim_matrix[i, j] = sim
125             self.user_sim_matrix[j, i] = sim # 矩阵对称
126     print("保存相似度矩阵到本地...")
```

```
122     dump(self.user_sim_matrix, self.matrix_file)
123
124     def predict_rating(self, u_idx, m_idx, k_neighbors):
125         """
126         预测指定用户对指定电影的评分 - 基于Top-K相似用户加权平均
127         u_idx: 用户矩阵下标
128         m_idx: 电影矩阵下标
129         返回预测分 (范围1-5)
130         """
131         if self.user_sim_matrix is None:
132             self.compute_similarity_matrix()
133         rated_by = np.where(self.rating_matrix[:, m_idx] > 0)[0]
134             # 所有对m_idx打过的用户下标
135         if len(rated_by) == 0:
136             return 0.0 # 没有邻居
137         sims = self.user_sim_matrix[u_idx, rated_by]
138             # 这些邻居与当前用户的相似度
139         top_k_idx = np.argsort(sims)[-k_neighbors:]
140             # 取Top-K相似邻居 (含负值, 后面会筛掉负
141             # 的)
142         neighbors = rated_by[top_k_idx]
143         neighbors_sims = sims[top_k_idx]
144         numerator = 0.0
145         denominator = 0.0
146         for n_idx, sim in zip(neighbors, neighbors_sims):
147             if sim <= 0: continue # 只用正向相似邻居
148             numerator += sim * self.rating_matrix[n_idx, m_idx]
149             denominator += abs(sim)
150         if denominator == 0:
151             return 0.0 # 没有有效邻居则0
152         predicted_rating = numerator / denominator
153         # 保证预测分在1-5之间
154         return max(1.0, min(5.0, predicted_rating))
155
156     def generate_all_predictions(self):
```

```
153     """
154     为全部用户全部电影生成评分预测矩阵（包含训练分和预测分）
155     """
156     print("开始生成全矩阵预测评分...")
157     num_users, num_movies = self.rating_matrix.shape
158     self.predicted_ratings = np.zeros_like(self.rating_matrix
159                                           )
160     for u_idx in range(num_users):
161         if u_idx % 100 == 0:
162             print(f"处理用户 {u_idx}/{num_users}")
163         for m_idx in range(num_movies):
164             if self.rating_matrix[u_idx, m_idx] > 0:
165                 # 已打分，拷贝真实分
166                 self.predicted_ratings[u_idx, m_idx] = self.
167                     rating_matrix[u_idx, m_idx]
168             else:
169                 # 未打分，才做协同过滤预测
170                 self.predicted_ratings[u_idx, m_idx] = self.
171                     predict_rating(u_idx, m_idx, k_neighbors)
172
173     def recommend_movies(self, user_id, n_movies):
174         """
175         为指定用户生成推荐（返回n部预测分最高的未评分电影id和分
176         数）
177         """
178         u_idx = self.user_id2index[user_id]
179         unrated_mask = self.rating_matrix[u_idx] == 0
180         pred_scores = self.predicted_ratings[u_idx][unrated_mask]
181             # 仅统计未评分处
182         movie_indices = np.where(unrated_mask)[0]
183         if len(pred_scores) < n_movies:
184             top_n_idx = np.argsort(pred_scores)[::-1]
185         else:
186             top_n_idx = np.argsort(pred_scores)[-n_movies:][::-1]
187         recommendations = []
```

```
183     for idx in top_n_idx:
184         movie_idx = movie_indices[idx]
185         movie_id = self.index_to_movie[movie_idx]
186         score = pred_scores[idx]
187         recommendations.append((movie_id, score))
188     return recommendations
189
190 def calculate_test_error(self, test_df):
191     """
192     计算测试集上SSE和RMSE等，用于系统效果评估，同时输出预测
193     csv
194     """
195     sse = 0.0
196     count = 0
197     predictions = []
198     for row in test_df.itertuples():
199         if row.userId in self.user_id2index and row.movieId
200         in self.movie_id2index:
201             u_idx = self.user_id2index[row.userId]
202             m_idx = self.movie_id2index[row.movieId]
203             pred_rating = self.predicted_ratings[u_idx, m_idx
204             ]
205             true_rating = row.rating
206             sse += (true_rating - pred_rating) ** 2
207             count += 1
208             predictions.append({
209                 'userId': row.userId,
210                 'movieId': row.movieId,
211                 'true_rating': true_rating,
212                 'predicted_rating': pred_rating
213             })
214     rmse = math.sqrt(sse / count) if count > 0 else float('
215     inf')
216     pred_df = pd.DataFrame(predictions)
217     pred_df.to_csv(f'test_predictions_{self.sim_mode}.csv',
```

```
        index=False)
214     return sse, rmse, count
215
216 def save_recommendations_to_csv(recommendations, filename="
recommendations.csv"):
217     """
218     将所有用户推荐写入CSV。
219     每行格式: userId, RecMovieId_1, Score_1, ... RecMovieId_5,
        Score_5
220     """
221     columns = ['userId']
222     for i in range(5):
223         columns.extend([f'RecMovieId_{i+1}', f'Score_{i+1}'])
224     rows = []
225     for user_id, recs in recommendations.items():
226         row = [user_id]
227         for i in range(5):
228             if i < len(recs):
229                 row.extend([recs[i][0], round(recs[i][1], 3)])
230             else:
231                 row.extend([None, None])
232         rows.append(row)
233     results_df = pd.DataFrame(rows, columns=columns)
234     results_df.to_csv(filename, index=False)
235     print(f"推荐结果已保存到 {filename}")
236
237 # ===== 4. 主流程 =====
238 print("参数列表:", f"k_neighbors={k_neighbors}, n_movies={
        n_movies}, sim_mode={sim_mode}\n")
239 # 1. 实例化推荐器
240 recommender = MovieRecommender(rating_matrix, user_id2index,
        movie_id2index, sim_mode=sim_mode)
241 # 2. 计算用户两两相似度
242 recommender.compute_similarity_matrix()
243 # 3. 预测所有评分
```

```
244 recommender.generate_all_predictions()
245
246 print("为所有用户生成推荐...")
247 all_recommendations = {}
248 for user_id in all_user_ids:
249     all_recommendations[user_id] = recommender.recommend_movies(
250         user_id, n_movies)
251 save_recommendations_to_csv(all_recommendations, filename=f"
252     recommendations_{sim_mode}.csv")
253 print("为所有用户生成推荐完毕...\n")
254
255 # 4. 测试集评估
256 sse, rmse, count = recommender.calculate_test_error(test_df)
257 print(f"\n测试集评估结果({sim_mode}):")
258 print(f"测试样本数: {count}")
259 print(f"SSE (误差平方和): {sse:.2f}")
260 print(f"RMSE: {rmse:.4f}")
```

Listing 10 UUCF

7 附录 B: UUCF+Minihash 源代码

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import math
6 import warnings
7 import random
8 from collections import defaultdict
9 from joblib import dump, load
10
11 warnings.filterwarnings("ignore")
12 random.seed(42)
13 np.random.seed(42)
14 # ===== 1. 参数设置 =====
15 k_neighbors = 10      # 选择最相似的Top-K邻居
16 n_movies = 5          # 每个用户推荐电影数
17 n_hash_functions = 100 # MinHash的哈希函数数量
18 rating_threshold = 2.5 # 二值化阈值
19
20 # ===== 2. 数据加载与预处理 =====
21 train_df = pd.read_csv("datasets/train_set.csv") # 加载训练集
22 test_df = pd.read_csv("datasets/test_set.csv")  # 加载测试集
23
24 # 收集所有用户与电影的id
25 all_user_ids = sorted(train_df['userId'].unique())
26 all_movie_ids = sorted(train_df['movieId'].unique())
27
28 # 创建用户/电影与下标的映射
29 user_id2index = {u: idx for idx, u in enumerate(all_user_ids)}
30 movie_id2index = {m: idx for idx, m in enumerate(all_movie_ids)}
31 num_users = len(all_user_ids)
32 num_movies = len(all_movie_ids)
33 print(f"用户数量: {num_users}")
```

```
34 print(f"电影数量: {num_movies}")
35
36 # 初始化 用户-电影评分矩阵
37 rating_matrix = np.zeros((num_users, num_movies), dtype=np.
    float32)
38 for row in train_df.itertuples():
39     u_idx = user_id2index[row.userId]
40     m_idx = movie_id2index[row.movieId]
41     rating_matrix[u_idx, m_idx] = row.rating
42
43 # ===== 3. MinHash类 =====
44 class MinHash:
45     def __init__(self, n_hash_functions=100):
46         """
47         初始化 MinHash 类, 生成多组随机哈希函数参数。
48         :param n_hash_functions: 哈希函数的数量, 决定了签名的长
            度, 也影响近似的准确性。
49         """
50         self.n_hash_functions = n_hash_functions
51         self.prime = 4294967311 # 一个大于特征空间的素数, 用于哈
            希
52         # 随机生成 n_hash_functions 个哈希函数参数 a 和 b
53         self.a = [random.randint(1, self.prime - 1) for _ in
            range(n_hash_functions)]
54         self.b = [random.randint(0, self.prime - 1) for _ in
            range(n_hash_functions)]
55
56     def compute_signature(self, binary_vector):
57         """
58         计算一个0-1特征向量的 MinHash 签名。
59         :param binary_vector: 输入为一个 numpy 的一维0-1数组, 1表
            示该特征(集合中的元素)出现。
60         :return: 返回该特征的minhash签名, 是一个长度为
            n_hash_functions 的 numpy 数组。
61         """
```

```
62     signature = []
63     # 找到所有为1的下标位置，相当于集合中的元素编号
64     ones_positions = np.where(binary_vector)[0]
65
66     for i in range(self.n_hash_functions):
67         if len(ones_positions) == 0:
68             # 空集，哈希为正无穷
69             min_hash = float('inf')
70         else:
71             # 对每个1的位置（即集合中的每个元素），计算第i个
              # 哈希函数的值，然后取最小值作为这一组哈希的
              # signature
72             hash_values = [(self.a[i] * x + self.b[i]) % self
              .prime for x in ones_positions]
73             min_hash = min(hash_values) if hash_values else
              float('inf')
74             signature.append(min_hash)
75     return np.array(signature)
76
77     def estimate_similarity(self, sig1, sig2):
78         """
79         通过比较两个对象的MinHash签名，估算它们的Jaccard相似度。
80         :param sig1: 第一个签名（由compute_signature生成）
81         :param sig2: 第二个签名
82         :return: Jaccard相似度的近似值（0到1之间的浮点数）
83         """
84         if len(sig1) == 0 or len(sig2) == 0:
85             # 空向量的情况，直接返回0
86             return 0.0
87         # 统计两个签名向量相等的位置占总长度的比例，作为Jaccard相
              # 似度的近似
88         return np.mean(sig1 == sig2)
89     # ===== 4. 推荐系统主类 =====
90     class MovieRecommender:
91         def __init__(self, rating_matrix, user_id2index,
```

```
movie_id2index):
92     self.rating_matrix = rating_matrix
93     self.user_id2index = user_id2index
94     self.movie_id2index = movie_id2index
95     self.user_sim_matrix = None
96     self.matrix_file = 'user_similarity_matrix_minhash.joblib'
97
98     # 索引反查功能
99     self.index_to_movie = {idx: movie_id for movie_id, idx in
100                             movie_id2index.items()}
101
102     self.index_to_user = {idx: user_id for user_id, idx in
103                             user_id2index.items()}
104
105     # MinHash相关初始化
106     self.minhash = MinHash(n_hash_functions)
107
108     # 生成二值化矩阵
109     self.binary_matrix = (rating_matrix > rating_threshold).
110                             astype(np.int32)
111
112     # 预计算所有用户的MinHash签名
113     self.user_signatures = {}
114     for u_idx in range(self.rating_matrix.shape[0]):
115         self.user_signatures[u_idx] = self.minhash.
116             compute_signature(self.binary_matrix[u_idx])
117
118 def compute_similarity_matrix(self):
119     """计算用户相似度矩阵（使用MinHash）"""
120     if os.path.exists(self.matrix_file):
121         print("从缓存加载用户相似度矩阵...")
122         self.user_sim_matrix = load(self.matrix_file)
123         return
124
125     print("计算用户相似度矩阵...")
126     num_users = self.rating_matrix.shape[0]
127     self.user_sim_matrix = np.zeros((num_users, num_users),
```

```
dtype=np.float32)

121
122 for i in range(num_users):
123     if i % 100 == 0:
124         print(f"计算进度: {i}/{num_users}")
125         sig_i = self.user_signatures[i]
126
127         for j in range(i + 1, num_users):
128             sig_j = self.user_signatures[j]
129             sim = self.minhash.estimate_similarity(sig_i,
130                                                     sig_j)
131             self.user_sim_matrix[i, j] = sim
132             self.user_sim_matrix[j, i] = sim
133
134         print("保存相似度矩阵...")
135         dump(self.user_sim_matrix, self.matrix_file)
136
137 def predict_rating(self, u_idx, m_idx, k_neighbors):
138     """预测用户u_idx对电影m_idx的评分"""
139     if self.user_sim_matrix is None:
140         self.compute_similarity_matrix()
141
142     rated_by = np.where(self.rating_matrix[:, m_idx] > 0)[0]
143     if len(rated_by) == 0:
144         return 0.0
145
146     sims = self.user_sim_matrix[u_idx, rated_by]
147     top_k_idx = np.argsort(sims)[-k_neighbors:]
148     neighbors = rated_by[top_k_idx]
149     neighbors_sims = sims[top_k_idx]
150
151     numerator = 0.0
152     denominator = 0.0
153     for n_idx, sim in zip(neighbors, neighbors_sims):
154         if sim <= 0:
```

```
154         continue
155         numerator += sim * self.rating_matrix[n_idx, m_idx]
156         denominator += sim
157
158     if denominator == 0:
159         return 0.0
160
161     predicted_rating = numerator / denominator
162     return max(1.0, min(5.0, predicted_rating))
163
164 def generate_all_predictions(self):
165     """为所有用户对所有电影生成预测评分矩阵"""
166     print("开始生成所有预测评分...")
167     num_users, num_movies = self.rating_matrix.shape
168     self.predicted_ratings = np.zeros_like(self.rating_matrix)
169
170     for u_idx in range(num_users):
171         if u_idx % 100 == 0:
172             print(f"处理用户 {u_idx}/{num_users}")
173         for m_idx in range(num_movies):
174             if self.rating_matrix[u_idx, m_idx] > 0:
175                 self.predicted_ratings[u_idx, m_idx] = self.
176                     rating_matrix[u_idx, m_idx]
177             else:
178                 self.predicted_ratings[u_idx, m_idx] = self.
179                     predict_rating(u_idx, m_idx, k_neighbors)
180
181 def recommend_movies(self, user_id, n_movies):
182     """为指定用户生成推荐"""
183     u_idx = self.user_id2index[user_id]
184     unrated_mask = self.rating_matrix[u_idx] == 0
185     pred_scores = self.predicted_ratings[u_idx][unrated_mask]
186     movie_indices = np.where(unrated_mask)[0]
```

```
186         if len(pred_scores) < n_movies:
187             top_n_idx = np.argsort(pred_scores)[::-1]
188         else:
189             top_n_idx = np.argsort(pred_scores)[-n_movies:][::-1]
190
191     recommendations = []
192     for idx in top_n_idx:
193         movie_idx = movie_indices[idx]
194         movie_id = self.index_to_movie[movie_idx]
195         score = pred_scores[idx]
196         recommendations.append((movie_id, score))
197     return recommendations
198
199     def calculate_test_error(self, test_df):
200         """在测试集上计算误差指标"""
201         sse = 0.0
202         count = 0
203         predictions = []
204
205         for row in test_df.itertuples():
206             if row.userId in self.user_id2index and row.movieId
207                in self.movie_id2index:
208                 u_idx = self.user_id2index[row.userId]
209                 m_idx = self.movie_id2index[row.movieId]
210                 pred_rating = self.predicted_ratings[u_idx, m_idx
211                    ]
212                 true_rating = row.rating
213                 sse += (true_rating - pred_rating) ** 2
214                 count += 1
215                 predictions.append({
216                     'userId': row.userId,
217                     'movieId': row.movieId,
218                     'true_rating': true_rating,
219                     'predicted_rating': pred_rating
220                 })
```

```
219
220     rmse = math.sqrt(sse / count) if count > 0 else float('
        inf')
221     pred_df = pd.DataFrame(predictions)
222     pred_df.to_csv('test_predictions_minhash.csv', index=
        False)
223     return sse, rmse, count
224
225 def save_recommendations_to_csv(recommendations, filename="
recommendations.csv"):
226     """保存推荐结果到CSV"""
227     columns = ['userId']
228     for i in range(5):
229         columns.extend([f'RecMovieId_{i+1}', f'Score_{i+1}'])
230
231     rows = []
232     for user_id, recs in recommendations.items():
233         row = [user_id]
234         for i in range(5):
235             if i < len(recs):
236                 row.extend([recs[i][0], round(recs[i][1], 3)])
237             else:
238                 row.extend([None, None])
239         rows.append(row)
240
241     results_df = pd.DataFrame(rows, columns=columns)
242     results_df.to_csv(filename, index=False)
243     print(f"推荐结果已保存到 {filename}")
244
245 # ===== 5. 主流程 =====
246 print(f"参数列表: k_neighbors={k_neighbors}, n_movies={n_movies},
        n_hash_functions={n_hash_functions}\n")
247 recommender = MovieRecommender(rating_matrix, user_id2index,
        movie_id2index)
248 recommender.compute_similarity_matrix()
```

```
249 recommender.generate_all_predictions()
250
251 print("为所有用户生成推荐...")
252 all_recommendations = {}
253 for user_id in all_user_ids:
254     all_recommendations[user_id] = recommender.recommend_movies(
255         user_id, n_movies)
256 save_recommendations_to_csv(all_recommendations, filename="
257     recommendations_minhash.csv")
258 print("为所有用户生成推荐完毕...\n")
259
260 # 测试集评估
261 sse, rmse, count = recommender.calculate_test_error(test_df)
262 print("\n测试集评估结果(MinHash):")
263 print(f"测试样本数: {count}")
264 print(f"SSE (误差平方和): {sse:.2f}")
265 print(f"RMSE: {rmse:.4f}")
```

Listing 11 UUCF+Minihash

8 附录 C: CB 源代码

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import math
5 import warnings
6 from copy import deepcopy
7 import joblib
8 import time
9 warnings.filterwarnings("ignore")
10
11 # 1. 读取电影基本信息
12 df1 = pd.read_csv("datasets/movies.csv") # 包含movieId和genres
      (类型信息)
13
14 # 2. 电影内容预处理类定义
15 class moviePreprocess(object):
16     def __init__(self, df):
17         self.df = df
18         self.categories = self.df['genres'].values # 所有电
      影类型字符串
19         self.movieId = self.df['movieId'].values # 所有电
      影ID
20
21 # 计算每种类型出现的总次数, 并构建电影到类型的映射
22 def calSum(self):
23     # 电影ID -> 类型列表 (如 ['action', 'adventure'])
24     self.idToCategory = {self.movieId[i] : self.categories[i
      ].lower().split('|') for i in range(len(self.movieId))
      }
25     self.categorySet = set() # 存所有出现过的类型
26     self.categoryAppearSum = {} # 记录每种类型在全部
      电影中出现的次数
27     for key, value in self.idToCategory.items():
```

```
28         for word in value:
29             self.categorySet.add(word) # 加入全集
30             if word in self.categoryAppearSum:
31                 self.categoryAppearSum[word] += 1
32             else:
33                 self.categoryAppearSum[word] = 1
34
35 # 构建TF-IDF特征矩阵 (电影-类型矩阵, 每行为一部电影, 每列为类型)
36 def getTfidfMat(self):
37     self.calSum() # 先统计类型全集和出现次数
38     categoryList = list(self.categorySet)
39     mapper = {categoryList[i] : i for i in range(len(
40         categoryList))} # 类型名 -> 列索引
41     tfidfMat = np.zeros(shape=(len(self.movieId), len(
42         categoryList)), dtype=np.float32)
43     length = len(self.movieId)
44     for i in range(len(self.movieId)): # 对每一部电影
45         for word in self.idToCategory[self.movieId[i]]: #
46             电影涉及的类型
47             # TF-IDF公式:  $1 + \log(\text{总电影数} / \text{类型出现次数})$ , 可
48             平衡“冷门类型权重更大”
49             tfidfMat[i][mapper[word]] = 1 + math.log((1 +
50                 length) / (1 + self.categoryAppearSum[word]))
51 # 各行单位化 (L2范数归一化, 便于做余弦相似)
52 for i in range(len(tfidfMat)):
53     tfidfMat[i] = np.divide(tfidfMat[i], np.sqrt(np.sum(
54         np.square(tfidfMat[i]))))
55 return tfidfMat
56
57 # 计算两个TF-IDF矩阵之间的余弦相似度 (用于两个电影内容的相似
58 性度量)
59 def getCosSimilarBetweenTwoMat(self, v1, v2):
60     v1 = np.array(v1)
61     v2 = np.array(v2)
```

```
55     # 余弦相似度公式：点积/模长
56     return np.dot(v1, v2.T) / (np.linalg.norm(v1, axis=1).
57                                reshape(-1, 1) * np.linalg.norm(v2, axis=1))
58
59     # 构建全体电影之间的内容相似度矩阵
60     def getSimilarityMat(self):
61         tfidfMat = self.getTfidfMat()
62         return self.getCosSimilarBetweenTwoMat(tfidfMat, tfidfMat
63                                                )
64
65     # 3. 加载或计算电影相似度矩阵（避免重复计算，节省时间）
66     similarity_matrix_path = 'similarity_matrix.joblib'
67     if os.path.exists(similarity_matrix_path):
68         print("加载已保存的相似度矩阵...")
69         similarityMatrix = joblib.load(similarity_matrix_path) # 加
70                                载已有特征
71     else:
72         print("计算电影相似度矩阵（这可能需要一些时间）...")
73         processor = moviePreprocess(df1)
74         similarityMatrix = processor.getSimilarityMat() # 重新计算内
75                                容相似度
76         print("保存相似度矩阵到文件以便后续使用...")
77         joblib.dump(similarityMatrix, similarity_matrix_path) # 缓
78                                存，便于下次直接用
79
80     # 4. 读取用户评分数据（训练集）
81     df2 = pd.read_csv("datasets/train_set.csv")
82     user, movie, score = df2['userId'].values, df2['movieId'].values,
83                            df2['rating'].values
84
85     # 构建用户评分字典 users = {用户id: {电影id: 评分}}
86     users = {}
87     for u, m, s in zip(user, movie, score):
88         users.setdefault(u, {})[m] = s
```

```
84 # 5. 内容推荐主类（推荐系统核心逻辑类）
85 class recommandByContent(object):
86     def __init__(self, movieSimMatrix, movieId, mapper, users, k):
87         :
88         self.movieSimMatrix = movieSimMatrix # 电影-电影内容相似
89         度矩阵（由TF-IDF+余弦得到）
90         self.movieId = movieId # 所有电影ID集合
91         self.mapper = mapper # 电影id到矩阵下标
92         的映射
93         self.users = users # 用户评分字典
94         self.k = k # 推荐Top-K
95         # 为每个用户预处理其已评分电影的indexes和得分，优化后续计算
96         self.user_movie_indices = {}
97         self.user_scores = {}
98         for userId in self.users:
99             movie_indices = [self.mapper[movie_id] for movie_id
100                             in self.users[userId]]
101             scores = np.array([score for score in self.users[
102                             userId].values()])
103             self.user_movie_indices[userId] = np.array(
104                 movie_indices)
105             self.user_scores[userId] = scores
106
107         # 根据内容相似度，预测一个用户对一部电影的评分
108         def scoreMovie(self, userID, movieID):
109             if userID not in self.users:
110                 return 0.0 # 用户没数据，返回0
111             movie_idx = self.mapper[movieID]
112             user_movie_indices = self.user_movie_indices[userID]
113             user_scores = self.user_scores[userID]
114             similarities = self.movieSimMatrix[movie_idx][
115                 user_movie_indices] # 当前电影 与 该用户已评分电影的
116                 相似度
117             positive_mask = similarities > 0 # 只取正相似度
```

```
110     if not np.any(positive_mask):
111         return 0.0 # 如果没有类似电影, 返回0
112     positive_similarities = similarities[positive_mask]
113     positive_scores = user_scores[positive_mask]
114     # 加权平均: 每部已评分电影得分 * 内容相似度, 再归一化
115     return np.sum(positive_scores * positive_similarities) /
116         np.sum(positive_similarities)
117
118 # 批量预测 (适合测试集批量预测或高效计算)
119 def batchScoreMovies(self, userIDs, movieIDs):
120     predictions = np.zeros(len(userIDs))
121     # 整理为“每个用户, 哪些电影”结构, 减少重复查找
122     user_groups = {}
123     for i, (user, movie) in enumerate(zip(userIDs, movieIDs)):
124         :
125         if user not in user_groups:
126             user_groups[user] = []
127             user_groups[user].append((i, movie))
128     for user, movie_indices in user_groups.items():
129         if user not in self.users:
130             continue # 用户无历史直接跳过
131         indices = [idx for idx, _ in movie_indices]
132         movies = [movie for _, movie in movie_indices]
133         user_movie_indices = self.user_movie_indices[user]
134         user_scores = self.user_scores[user]
135         for i, movie in enumerate(movies):
136             movie_idx = self.mapper[movie]
137             similarities = self.movieSimMatrix[movie_idx][
138                 user_movie_indices]
139             positive_mask = similarities > 0
140             if np.any(positive_mask):
141                 positive_similarities = similarities[
142                     positive_mask]
143                 positive_scores = user_scores[positive_mask]
144                 predictions[indices[i]] = np.sum(
```

```
        positive_scores * positive_similarities) /  
        np.sum(positive_similarities)  
141     return predictions  
142  
143     # 针对某用户，对其未看（未评分）过的所有电影进行预测打分  
144     def recommandSingle(self, peopleId):  
145         if peopleId not in self.users:  
146             # 冷启动用户，可能全为0  
147             return {id: 0.0 for id in self.movieId}  
148         finalScore = {}  
149         user Rated_movies = set(self.users[peopleId].keys())  
150         user_movie_indices = self.user_movie_indices[peopleId]  
151         user_scores = self.user_scores[peopleId]  
152         unrated_movies = [id for id in self.movieId if id not in  
            user Rated_movies] # 未看过  
153         unrated_indices = np.array([self.mapper[id] for id in  
            unrated_movies])  
154         batch_size = 500 # 分批防止内存爆炸  
155         for i in range(0, len(unrated_movies), batch_size):  
156             batch_ids = unrated_movies[i:i+batch_size]  
157             batch_indices = unrated_indices[i:i+batch_size]  
158             for j, movie_id in enumerate(batch_ids):  
159                 movie_idx = batch_indices[j]  
160                 similarities = self.movieSimMatrix[movie_idx][  
                    user_movie_indices]  
161                 positive_mask = similarities > 0  
162                 if np.any(positive_mask):  
163                     positive_similarities = similarities[  
                        positive_mask]  
164                     positive_scores = user_scores[positive_mask]  
165                     score = np.sum(positive_scores *  
                        positive_similarities) / np.sum(  
                            positive_similarities)  
166                     finalScore[movie_id] = score  
167                 else:
```

```
168         finalScore[movie_id] = 0.0
169     return finalScore
170
171     # 输出单个用户未评分电影按内容推荐得分排序后的前K个结果
172     def recommandSpecificPerson(self, userID):
173         finalScore = self.recommandSingle(userID) # {电影id: 预
            测分}
174         predictScore = deepcopy(finalScore)
175         finalScore = sorted(finalScore.items(), key=lambda x:x
            [1], reverse=True)
176         return predictScore, finalScore[: min(self.k, len(
            finalScore))]
177
178     # 6. 建立电影id到下标的映射 (便于矩阵查找)
179     movieId = df1['movieId'].values
180     mapper = {movieId[i]: i for i in range(len(movieId))}
181
182     # 7. 推荐系统实例化
183     print("初始化推荐系统...")
184     recommender = recommandByContent(similarityMatrix, movieId,
        mapper, users, 5)
185
186     # 8. 读取测试集, 用于评估推荐效果
187     testData = pd.read_csv("datasets/test_set.csv")
188     testUsers, testMovies, rating = testData['userId'].values,
        testData['movieId'].values, testData['rating'].values
189
190     # 对测试集进行批量推荐评分预测
191     def testResult(testUsers, testMovies, recommender):
192         return recommender.batchScoreMovies(testUsers, testMovies)
193
194     print("开始测试集预测...")
195     start_time = time.time()
196     predict = testResult(testUsers, testMovies, recommender)
197     end_time = time.time()
```

```
198 print(f"测试集预测完成, 耗时: {end_time - start_time:.2f} 秒")
199
200 # 计算预测与真实分差的残差平方和 (SSE)
201 SSE = np.sum(np.square(np.array(predict) - rating))
202 print(f"测试集残差平方和(SSE): {SSE}")
203
204 # ===== 预测所有用户对所有未评分电影的内容推荐得分, 输出完整打分
    表 =====
205 all_users = set(df2['userId']).union(set(testData['userId']))
206 all_movies = set(df1['movieId'])
207 # 构建测试集中真实评分查表
208 test_true_rating = {(row['userId'], row['movieId']): row['rating']
    ] for _, row in testData.iterrows()}
209
210 print("开始处理所有用户对所有未评分电影的预测...")
211 start_time = time.time()
212 output_list = []
213 batch_size = 1000 # 分批预测加速
214
215 for user in sorted(all_users):
216     print(f"处理用户 {user}...")
217     rated_movies = set(users[user].keys()) if user in users else
        set()
218     unrated_movies = [movie for movie in sorted(all_movies) if
        movie not in rated_movies]
219     for i in range(0, len(unrated_movies), batch_size):
220         batch_movies = unrated_movies[i:i+batch_size]
221         if user in users: # 若有历史行为, 调用预测接口
222             batch_users = np.array([user] * len(batch_movies))
223             batch_predictions = recommender.batchScoreMovies(
                batch_users, np.array(batch_movies))
224             for j, movie in enumerate(batch_movies):
225                 # 取真实分 (只有测试集有), 否则填空
226                 true_rating = test_true_rating.get((user, movie),
                    '')
```

```
227         output_list.append([user, movie, true_rating,
228                               batch_predictions[j]])
229     else:
230         # 冷启动用户, 全部为0
231         for movie in batch_movies:
232             true_rating = test_true_rating.get((user, movie),
233                                                 '')
234             output_list.append([user, movie, true_rating,
235                                 0.0])
236
237 end_time = time.time()
238 print(f"预测完成, 总耗时: {end_time - start_time:.2f} 秒")
239 output_df = pd.DataFrame(output_list, columns=['userId', 'movieId',
240         'true_rating', 'predicted_rating'])
241 output_df.to_csv('user_unrated_predict.csv', index=False)
242 print("所有用户对所有未评分电影的评分及真实分已保存到
243       user_unrated_predict.csv")
244
245 # ===== 输出每个用户内容推荐得分最高的前n部电影 =====
246 def recommend_topn_to_all_users(recommender, all_users, n,
247     save_path):
248     result_rows = []
249     for user in sorted(all_users):
250         # 得到该用户未评分电影的预测分
251         predict_scores = recommender.recommendSingle(user)
252         # 按预测分数降序排列, 选前n
253         top_n_movies = sorted(predict_scores.items(), key=lambda
254             x: x[1], reverse=True)[:n]
255         # 若数量不足n, 补空值
256         while len(top_n_movies) < n:
257             top_n_movies.append(('', ''))
258         # 构造输出行: userId, movieId_1, score_1, ..., movieId_n,
259             score_n
260         row = [user]
261         for movie_id, score in top_n_movies:
```

```
254         row.append(movie_id)
255         row.append(round(score, 4) if score != '' else '')
256     result_rows.append(row)
257
258     # 构造表头字段
259     col_names = ['userId']
260     for i in range(1, n+1):
261         col_names.extend([f'movieId_{i}', f'score_{i}'])
262
263     pd.DataFrame(result_rows, columns=col_names).to_csv(save_path
264         , index=False, encoding='utf-8-sig')
265     print(f"每个用户前{n}部推荐已保存到 {save_path}")
266
267     # 使用方法示例：每人推荐n部
268     n = 5 # 推荐Top-n
269     output_csv_path = f"user_top{n}_recommend.csv"
270     recommend_topn_to_all_users(recommender, all_users, n,
271         output_csv_path)
```

Listing 12 CB

9 附录 D: CB+Minihash 源代码

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import math
5 import warnings
6 from copy import deepcopy
7 import joblib
8 import time
9 from datasketch import MinHash
10 from joblib import Parallel, delayed
11
12 warnings.filterwarnings("ignore") # 忽略所有警告
13
14 # 1. 读取电影基本信息
15 df1 = pd.read_csv("datasets/movies.csv") # movieId, genres
16
17 # 2. MiniHash+Jaccard内容预处理类 (并行加速)
18 class moviePreprocess(object):
19     def __init__(self, df, num_perm=128):
20         self.df = df
21         self.categories = self.df['genres'].values
22         self.movieId = self.df['movieId'].values
23         self.num_perm = num_perm
24
25     def getBinaryMat(self):
26         self.idToCategory = {self.movieId[i]: set(self.categories
27             [i].lower().split('|')) for i in range(len(self.
28                 movieId))}
29         self.categorySet = set()
30         for v in self.idToCategory.values():
31             self.categorySet.update(v)
32         categoryList = list(self.categorySet)
33         mapper = {categoryList[i]: i for i in range(len(
```

```
        categoryList)))}
32     binaryMat = np.zeros((len(self.movieId), len(categoryList
        )), dtype=np.uint8)
33     for i, mId in enumerate(self.movieId):
34         for word in self.idToCategory[mId]:
35             binaryMat[i][mapper[word]] = 1
36     self.categoryList = categoryList
37     return binaryMat
38
39 def getMinihashes(self):
40     binaryMat = self.getBinaryMat()
41     minhashes = []
42     for row in binaryMat:
43         m = MinHash(num_perm=self.num_perm)
44         for idx, v in enumerate(row):
45             if v == 1:
46                 m.update(str(self.categoryList[idx]).encode('
                        utf8'))
47                 minhashes.append(m)
48     return minhashes
49
50 def getSimilarityMat(self):
51     minhashes = self.getMinihashes()
52     n = len(minhashes)
53     sim_mat = np.zeros((n, n), dtype=np.float32)
54
55     def fill_row(i):
56         row = np.zeros(n, dtype=np.float32)
57         for j in range(i, n):
58             row[j] = minhashes[i].jaccard(minhashes[j])
59         return row
60
61     print("并行计算相似度矩阵 (MiniHash+Jaccard) ...")
62     rows = Parallel(n_jobs=-1, prefer="threads")(delayed(
        fill_row)(i) for i in range(n))
```

```
63         for i, row in enumerate(rows):
64             sim_mat[i, i:] = row[i:]
65             sim_mat[i:, i] = row[i:]
66         return sim_mat
67
68 # 3. 缓存加载/计算相似度矩阵
69 similarity_matrix_path = 'similarity_minihash_matrix.joblib'
70 if os.path.exists(similarity_matrix_path):
71     print("加载已保存的相似度矩阵...")
72     similarityMatrix = joblib.load(similarity_matrix_path)
73 else:
74     print("采用Minihash+Jaccard相似度，正在并行计算内容相似度矩阵\n(这可能需要一些时间) ...")
75     processor = moviePreprocess(df1, num_perm=128)
76     similarityMatrix = processor.getSimilarityMat()
77     print("保存相似度矩阵到文件以便后续使用...")
78     joblib.dump(similarityMatrix, similarity_matrix_path)
79
80 # 4. 读取评分训练集
81 df2 = pd.read_csv("datasets/train_set.csv")
82 user, movie, score = df2['userId'].values, df2['movieId'].values,
83     df2['rating'].values
84
85 # 构建用户历史评分字典:
86 users = {}
87 for u, m, s in zip(user, movie, score):
88     users.setdefault(u, {})[m] = s
89
90 # 5. 推荐主类（原版不变，兼容迷你哈希相似度）
91 class recommanByContent(object):
92     def __init__(self, movieSimMatrix, movieId, mapper, users, k)
93         :
94             self.movieSimMatrix = movieSimMatrix
95             self.movieId = movieId
96             self.mapper = mapper
```

```
95     self.users = users
96     self.k = k
97     self.user_movie_indices = {}
98     self.user_scores = {}
99     for userId in self.users:
100         movie_indices = [self.mapper[movie_id] for movie_id
101                           in self.users[userId]]
102         scores = np.array([score for score in self.users[
103                             userId].values()])
104         self.user_movie_indices[userId] = np.array(
105             movie_indices)
106         self.user_scores[userId] = scores
107
108     def scoreMovie(self, userID, movieID):
109         if userID not in self.users:
110             return 0.0
111         movie_idx = self.mapper[movieID]
112         user_movie_indices = self.user_movie_indices[userID]
113         user_scores = self.user_scores[userID]
114         similarities = self.movieSimMatrix[movie_idx][
115             user_movie_indices]
116         positive_mask = similarities > 0
117         if not np.any(positive_mask):
118             return 0.0
119         positive_similarities = similarities[positive_mask]
120         positive_scores = user_scores[positive_mask]
121         return np.sum(positive_scores * positive_similarities) /
122             np.sum(positive_similarities)
123
124     def batchScoreMovies(self, userIDs, movieIDs):
125         predictions = np.zeros(len(userIDs))
126         user_groups = {}
127         for i, (user, movie) in enumerate(zip(userIDs, movieIDs)):
128             :
129             if user not in user_groups:
```

```
124         user_groups[user] = []
125         user_groups[user].append((i, movie))
126     for user, movie_indices in user_groups.items():
127         if user not in self.users:
128             continue
129         indices = [idx for idx, _ in movie_indices]
130         movies = [movie for _, movie in movie_indices]
131         user_movie_indices = self.user_movie_indices[user]
132         user_scores = self.user_scores[user]
133         for i, movie in enumerate(movies):
134             movie_idx = self.mapper[movie]
135             similarities = self.movieSimMatrix[movie_idx][
136                 user_movie_indices]
137             positive_mask = similarities > 0
138             if np.any(positive_mask):
139                 positive_similarities = similarities[
140                     positive_mask]
141                 positive_scores = user_scores[positive_mask]
142                 predictions[indices[i]] = np.sum(
143                     positive_scores * positive_similarities) /
144                     np.sum(positive_similarities)
145         return predictions
146
147     def recommandSingle(self, peopleId):
148         if peopleId not in self.users:
149             return {id: 0.0 for id in self.movieId}
150         finalScore = {}
151         user Rated movies = set(self.users[peopleId].keys())
152         user_movie_indices = self.user_movie_indices[peopleId]
153         user_scores = self.user_scores[peopleId]
154         unrated_movies = [id for id in self.movieId if id not in
155                             user Rated movies]
156         unrated_indices = np.array([self.mapper[id] for id in
157                                     unrated_movies])
158         batch_size = 500
```

```
153     for i in range(0, len(unrated_movies), batch_size):
154         batch_ids = unrated_movies[i:i+batch_size]
155         batch_indices = unrated_indices[i:i+batch_size]
156         for j, movie_id in enumerate(batch_ids):
157             movie_idx = batch_indices[j]
158             similarities = self.movieSimMatrix[movie_idx][
                user_movie_indices]
159             positive_mask = similarities > 0
160             if np.any(positive_mask):
161                 positive_similarities = similarities[
                    positive_mask]
162                 positive_scores = user_scores[positive_mask]
163                 score = np.sum(positive_scores *
                    positive_similarities) / np.sum(
                    positive_similarities)
164                 finalScore[movie_id] = score
165             else:
166                 finalScore[movie_id] = 0.0
167         return finalScore
168
169     def recommandSpecificPerson(self, userID):
170         finalScore = self.recommandSingle(userID)
171         predictScore = deepcopy(finalScore)
172         finalScore = sorted(finalScore.items(), key=lambda x:x
            [1], reverse=True)
173         return predictScore, finalScore[: min(self.k, len(
            finalScore))]
174
175 # 6. 建立电影ID到矩阵下标映射
176 movieId = df1['movieId'].values
177 mapper = {movieId[i]: i for i in range(len(movieId))}
178
179 print("初始化推荐系统...")
180 recommender = recommandByContent(similarityMatrix, movieId,
    mapper, users, 5)
```

```
181
182 # 7. 读取测试集验证
183 testData = pd.read_csv("datasets/test_set.csv")
184 testUsers, testMovies, rating = testData['userId'].values,
    testData['movieId'].values, testData['rating'].values
185
186 def testResult(testUsers, testMovies, recommender):
187     return recommender.batchScoreMovies(testUsers, testMovies)
188
189 print("开始测试集预测...")
190 start_time = time.time()
191 predict = testResult(testUsers, testMovies, recommender)
192 end_time = time.time()
193 print(f"测试集预测完成, 耗时: {end_time - start_time:.2f} 秒")
194
195 SSE = np.sum(np.square(np.array(predict) - rating))
196 print(f"测试集残差平方和(SSE): {SSE}")
197
198 # 8. 预测所有用户所有未评分电影
199 all_users = set(df2['userId']).union(set(testData['userId']))
200 all_movies = set(df1['movieId'])
201 test_true_rating = {(row['userId'], row['movieId']): row['rating']
    for _, row in testData.iterrows()}
202
203 print("开始处理所有用户对所有未评分电影的预测...")
204 start_time = time.time()
205 output_list = []
206 batch_size = 1000
207
208 for user in sorted(all_users):
209     print(f"处理用户 {user}...")
210     rated_movies = set(users[user].keys()) if user in users else
        set()
211     unrated_movies = [movie for movie in sorted(all_movies) if
        movie not in rated_movies]
```

```
212     for i in range(0, len(unrated_movies), batch_size):
213         batch_movies = unrated_movies[i:i+batch_size]
214         if user in users:
215             batch_users = np.array([user] * len(batch_movies))
216             batch_predictions = recommender.batchScoreMovies(
                batch_users, np.array(batch_movies))
217             for j, movie in enumerate(batch_movies):
218                 true_rating = test_true_rating.get((user, movie),
                    '')
219                 output_list.append([user, movie, true_rating,
                    batch_predictions[j]])
220         else:
221             for movie in batch_movies:
222                 true_rating = test_true_rating.get((user, movie),
                    '')
223                 output_list.append([user, movie, true_rating,
                    0.0])
224
225 end_time = time.time()
226 print(f"预测完成, 总耗时: {end_time - start_time:.2f} 秒")
227 output_df = pd.DataFrame(output_list, columns=['userId', 'movieId',
    'true_rating', 'predicted_rating'])
228 output_df.to_csv('user_unrated_minihash_predict.csv', index=False)
229
230 print("所有用户对所有未评分电影的评分及真实分已保存到
    user_unrated_minihash_predict.csv")
231
232 # 9. 推荐TopN输出
233 def recommend_topn_to_all_users(recommender, all_users, n,
    save_path):
234     result_rows = []
235     for user in sorted(all_users):
236         predict_scores = recommender.recommendSingle(user)
237         top_n_movies = sorted(predict_scores.items(), key=lambda
            x: x[1], reverse=True)[:n]
```

```
237     while len(top_n_movies) < n:
238         top_n_movies.append(('', ''))
239     row = [user]
240     for movie_id, score in top_n_movies:
241         row.append(movie_id)
242         row.append(round(score, 4) if score != '' else '')
243     result_rows.append(row)
244     col_names = ['userId']
245     for i in range(1, n+1):
246         col_names.extend([f'movieId_{i}', f'score_{i}'])
247     pd.DataFrame(result_rows, columns=col_names).to_csv(save_path
248         , index=False, encoding='utf-8-sig')
249     print(f"每个用户前{n}部推荐已保存到 {save_path}")
250 n = 5
251 output_csv_path = f"user_top{n}_minihash_recommend.csv"
252 recommend_topn_to_all_users(recommender, all_users, n,
    output_csv_path)
```

Listing 13 CB+Minihash